

## 第二章

(第 13、14、15、16、17、18、21、30、31、34、35 题)

13. 已知 C 语言中的按位异或运算 (“XOR”) 用符号 “^” 表示。对于任意一个位序列  $a$ ,  $a \wedge a = 0$ , C 语言程序可以利用这个特性来实现两个数值交换的功能。以下是一个实现该功能的 C 语言函数:

```
1 void xor_swap(int *x, int *y)
2 {
3     *y = *x ^ *y; /* 第一步 */
4     *x = *x ^ *y; /* 第二步 */
5     *y = *x ^ *y; /* 第三步 */
6 }
```

假定执行该函数时  $*x$  和  $*y$  的初始值分别为  $a$  和  $b$ , 即  $*x = a$  且  $*y = b$ , 请给出每一步执行结束后,  $x$  和  $y$  各自指向的内存单元中的内容分别是什么?

参考答案:

第一步结束后,  $x$  和  $y$  指向的内存单元内容各为  $a$  和  $a \wedge b$

第二步结束后,  $x$  和  $y$  指向的内存单元内容各为  $b$  和  $a \wedge b$

第三步结束后,  $x$  和  $y$  指向的内存单元内容各为  $b$  和  $a$

14. 假定某个实现数组元素倒置的函数 `reverse_array` 调用了第 13 题中给出的 `xor_swap` 函数:

```
1 void reverse_array(int a[], int len)
2 {
3     int left, right = len - 1;
4     for (left = 0; left <= right; left++, right--)
5         xor_swap(&a[left], &a[right]);
6 }
```

当  $len$  为偶数时, `reverse_array` 函数的执行没有问题。但是, 当  $len$  为奇数时, 函数的执行结果不正确。请问, 当  $len$  为奇数时会出现什么问题? 最后一次循环中的  $left$  和  $right$  各取什么值? 最后一次循环中调用 `xor_swap` 函数后的返回值是什么? 对 `reverse_array` 函数作怎样的改动就可消除该问题?

参考答案:

当  $len$  为奇数时, 最后一次循环执行的是将最中间的数与自己进行交换, 即  $left$  和  $right$  都指向最中间数组元素, 因而在调用 `xor_swap` 函数过程中的每一步执行  $*x \wedge *y$  时结果都是 0, 并将 0 写入了最中间的数组元素, 从而改变了原来的数值。

可以将 `for` 循环中的终止条件改为 “ $left < right$ ”, 这样, 在  $len$  为奇数时最中间的数组元素不动。

15. 假设以下表 2.13 中的  $x$  和  $y$  是某 C 语言程序中的 `char` 型变量, 请根据 C 语言中的按位运算和逻辑运算的定义, 填写表 2.13, 要求用十六进制形式填写。

表 2.13 题 15 用表

| $x$  | $y$  | $x \wedge y$ | $x \& y$ | $x   y$ | $\sim x   \sim y$ | $x \& !y$ | $x \& \& y$ | $x    y$ | $!x    !y$ | $x \& \& \sim y$ |
|------|------|--------------|----------|---------|-------------------|-----------|-------------|----------|------------|------------------|
| 0x5F | 0xA0 |              |          |         |                   |           |             |          |            |                  |
| 0xC7 | 0xF0 |              |          |         |                   |           |             |          |            |                  |
| 0x80 | 0x7F |              |          |         |                   |           |             |          |            |                  |
| 0x07 | 0x55 |              |          |         |                   |           |             |          |            |                  |

参考答案:

| x    | y    | x^y  | x&y  | x y  | ~x ~y | x&!y | x&&y | x  y | !x  !y | x&&~y |
|------|------|------|------|------|-------|------|------|------|--------|-------|
| 0x5F | 0xA0 | 0xFF | 0x00 | 0xFF | 0xFF  | 0x00 | 0x01 | 0x01 | 0x00   | 0x01  |
| 0xC7 | 0xF0 | 0x37 | 0xC0 | 0xF7 | 0x3F  | 0x00 | 0x01 | 0x01 | 0x00   | 0x01  |
| 0x80 | 0x7F | 0xFF | 0x00 | 0xFF | 0xFF  | 0x00 | 0x01 | 0x01 | 0x00   | 0x01  |
| 0x07 | 0xFF | 0x07 | 0x07 | 0xFF | 0xF8  | 0x00 | 0x01 | 0x01 | 0x00   | 0x00  |

16. 对于一个  $n$  ( $n \geq 8$ ) 位的变量  $x$ , 请根据 C 语言中按位运算的定义, 写出满足下列要求的 C 语言表达式。

- (1)  $x$  的最高有效字节不变, 其余各位全变为 0。
- (2)  $x$  的最低有效字节不变, 其余各位全变为 0。
- (3)  $x$  的最低有效字节全变为 0, 其余各位取反。
- (4)  $x$  的最低有效字节全变 1, 其余各位不变。

参考答案:

- (1)  $(x >> (n-8)) << (n-8)$
- (2)  $x \& 0xFF$
- (3)  $((x \wedge \sim 0xFF) >> 8) << 8$
- (3)  $x | 0xFF$

17. 以下是一个由反汇编器生成的一段针对某个小端方式处理器的机器级代码表示文本, 其中, 最左边是指令所在的存储单元地址, 冒号后面是指令的机器码, 最右边是指令的汇编语言表示, 即汇编指令。已知反汇编输出中的机器数都采用补码表示, 请给出指令代码中划线部分表示的机器数对应的真值。

```

80483d2: 81 ec b8 01 00 00    sub    &0x1b8, %esp
80483d8: 8b 55 08                mov    0x8(%ebp), %edx
80483db: 83 c2 14              add    $0x14, %edx
80483de: 8b 85 58 fe ff ff     mov    0xfffffe58(%ebp), %eax
80483e4: 03 02                  add    (%edx), %eax
80483e6: 89 85 74 fe ff ff     mov    %eax, 0xfffffe74(%ebp)
80483ec: 8b 55 08                mov    0x8(%ebp), %edx
80483ef: 83 c2 44              add    $0x44, %edx
80483f2: 8b 85 c8 fe ff ff     mov    0xfffffec8(%ebp), %eax
80483f8: 89 02                  mov    %eax, (%edx)
80483fa: 8b 45 10              mov    0x10(%ebp), %eax
80483fd: 03 45 0c              add    0xc(%ebp), %eax
8048400: 89 85 ec fe ff ff     mov    %eax, 0xfffffeec(%ebp)
8048406: 8b 45 08                mov    0x8(%ebp), %eax
8048409: 83 c0 20              add    $0x20, %eax

```

参考答案:

- b8 01 00 00: 机器数为 000001B8H, 真值为 +1 1011 1000B = 440
- 14: 机器数为 14H, 真值为 +1 0100B = 20
- 58 fe ff ff: 机器数为 FFFFE58H, 真值为 -1 1010 1000B = -424
- 74 fe ff ff: 机器数为 FFFFE74H, 真值为 -1 1000 1100B = -396
- 44: 机器数为 44H, 真值为 +100 0100B = 68
- c8 fe ff ff: 机器数为 FFFFE8H, 真值为 -1 1000 1100B = -312
- 10: 机器数为 10H, 真值为 +1000B = 16
- 0c: 机器数为 0CH, 真值为 +1100B = 12

**ec fe ff ff:** 机器数为 FFFFFEECH, 真值为-1 0001 0100B = -276

**20:** 机器数为 20H, 真值为+00100000B=32

18. 假设以下 C 语言函数 `compare_str_len` 用来判断两个字符串的长度, 当字符串 `str1` 的长度大于 `str2` 的长度时函数返回值为 1, 否则为 0。

```
1 int compare_str_len(char *str1, char *str2)
2 {
3     return strlen(str1) - strlen(str2) > 0;
4 }
```

已知 C 语言标准库函数 `strlen` 原型声明为“`size_t strlen(const char *s);`”, 其中, `size_t` 被定义为 `unsigned int` 类型。请问: 函数 `compare_str_len` 在什么情况下返回的结果不正确? 为什么? 为使函数正确返回结果应如何修改代码?

参考答案:

因为 `size_t` 被定义为 `unsigned int` 类型, 因此, 库函数 `strlen` 的返回值为无符号整数。函数 `compare_str_len` 中的返回值是 `strlen(str1) - strlen(str2) > 0`, 这个关系表达式中 `>` 号左边是两个无符号数相减, 其差还是无符号整数, 因而总是大于等于 0, 也即在 `str1` 的长度小于 `str2` 的长度时结果也为 1。显然, 这是错误的。

只要将第 3 行语句改为以下形式即可:

```
3     return strlen(str1) > strlen(str2);
```

21. 以下是两段 C 语言代码, 函数 `arith()` 是直接写用 C 语言写的, 而 `optarith()` 是对 `arith()` 函数以某个确定的 `M` 和 `N` 编译生成的机器代码反编译生成的。根据 `optarith()`, 可以推断函数 `arith()` 中 `M` 和 `N` 的值各是多少?

```
#define M
#define N
int arith(int x, int y)
{
    int result = 0;
    result = x*M + y/N;
    return result;
}

int optarith(int x, int y)
{
    int t = x;
    x <<= 4;
    x -= t;
    if (y < 0) y += 3;
    y >>= 2;
    return x+y;
}
```

参考答案:

可以看出 `x*M` 和 “`int t = x; x <<= 4; x -= t;`” 三句对应, 这些语句实现了 `x` 乘 15 的功能 (左移 4 位相当于乘以 16, 然后再减 1), 因此, `M` 等于 15;

`y/N` 与 “`if (y < 0) y += 3; y >>= 2;`” 两句对应, 第二句 “`y` 右移 2 位” 实现了 `y` 除以 4 的功能, 因此 `N` 是 4。而第一句 “`if (y < 0) y += 3;`” 主要用于对 `y = -1` 时进行调整, 若不调整, 则 `-1 >> 2 = -1` 而 `-1/4 = 0`, 两者不等; 调整后 `-1+3=2`, `2 >> 2 = 0`, 两者相等。

30. 在字长为 32 位的计算机上, 有一个函数其原型声明为 “`int ch_mul_overflow(int x, int y);`”, 该函数用

于对两个 int 型变量 x 和 y 的乘积判断是否溢出，若溢出则返回 1，否则返回 0。请使用 64 位精度的整数类型 long long 来编写该函数。

参考答案：

使用 64 位精度的乘法实现两个 32 位带符号整数相乘（专门的补码乘法运算），可以通过乘积的高 32 位和低 32 位的关系来进行溢出判断。判断规则是：若高 32 位中每一位都与低 32 位的最高位相同，则不溢出；否则溢出。

```
1  int ch_mul_overflow(int x, int y)
2  {
3      long long prod_64= (long long) x*y;
4      return prod_64 != (int) prod_64;
5  }
```

第 3 行赋值语句的右边采用强制类型转换，使得 x 和 y 相乘的结果强制以 64 位乘积的形式保留在 64 位 long long 型变量 prod\_64 中。在第 4 行关系运算符!=右边的强制类型转换，将一个 64 位乘积的高 32 位丢弃，然后，在进行关系运算时，因为关系运算符!=的左边是一个 64 位整数，所以，右边的 32 位数必须再进行符号扩展以转换为 64 位整数，然后再与左边的整数进行比较。若乘积没有溢出，则丢弃的高 32 位和后面符号扩展的 32 位相同，因而比较结果一定是相等，返回为 0；若乘积有溢出，则比较结果一定不相等，返回为 1。

若第 3 行赋值语句改成如下形式，则 prod\_64 得到的是低 32 位乘积进行符号扩展后的 64 位值，因此，当结果溢出时，prod\_64 中得到的并不是正确的 64 位乘积。

```
3      long long prod_64= x*y;
```

31. 对于第 2.7.5 节中例 2.31 存在的整数溢出漏洞，如果将其中的第 5 行改为以下两个语句：

```
unsigned long long arraysize=count*(unsigned long long)sizeof(int);
```

```
int *myarray = (int *) malloc(arraysize);
```

已知 C 语言标准库函数 malloc 的原型声明为“void \*malloc(size\_t size);”，其中，size\_t 定义为 unsigned int 类型，则上述改动能否消除整数溢出漏洞？若能则说明理由；若不能则给出修改方案。

参考答案：

上述改动无法消除整数溢出漏洞，这种改动方式虽然使得 arraysize 的表示范围扩大了，避免了 arraysize 的溢出，不过，当调用 malloc 函数时，若 arraysize 的值大于 32 位的 unsigned int 的最大可表示值，则 malloc 函数还是只能按 32 位数给出的值去申请空间，同样会发生整数溢出漏洞。

程序应该在调用 malloc 函数之前检测所申请的空间大小是否大于 32 位无符号整数的可表示范围，若是，则返回-1，表示不成功；否则再申请空间并继续进行数组复制。修改后的程序如下：

```
1  /* 复制数组到堆中，count 为数组元素个数 */
2  int copy_array(int *array, int count) {
3      int i;
4      /* 在堆区申请一块内存 */
5      unsigned long long arraysize=count*(unsigned long long)sizeof(int);
6      size_t myarraysize=(size_t) arraysize;
7      if (myarraysize!=arraysize)
8          return -1;
9      int *myarray = (int *) malloc(myarraysize);
10     if (myarray == NULL)
11         return -1;
12     for (i = 0; i < count; i++)
```

```

13         myarray[i] = array[i];
14     return count;
15 }

```

34. 无符号整数变量  $ux$  和  $uy$  的声明和初始化如下：

```
unsigned ux=x;
```

```
unsigned uy=y;
```

若  $\text{sizeof}(\text{int})=4$ ，则对于任意  $\text{int}$  型变量  $x$  和  $y$ ，判断以下关系表达式是否永真。若永真则给出证明；若不永真则给出结果为假时  $x$  和  $y$  的取值。

- |                                               |                                       |
|-----------------------------------------------|---------------------------------------|
| (1) $(x*x) \geq 0$                            | (2) $(x-1 < 0) \parallel x > 0$       |
| (3) $x < 0 \parallel -x \leq 0$               | (4) $x > 0 \parallel -x \geq 0$       |
| (5) $x \& 0xf \neq 15 \parallel (x < 28) < 0$ | (6) $x > y == (-x < -y)$              |
| (7) $\sim x + \sim y == \sim(x+y)$            | (8) $(\text{int})(ux-uy) == -(y-x)$   |
| (9) $((x >> 2) < 2) \leq x$                   | (10) $x*4 + y*8 == (x < 2) + (y < 3)$ |
| (11) $x/4 + y/8 == (x >> 2) + (y >> 3)$       | (12) $x*y == ux*uy$                   |
| (13) $x+y == ux+uy$                           | (14) $x*\sim y + ux*uy == -x$         |

参考答案：

(1)  $(x*x) \geq 0$

非永真。例如， $x=65\ 534$  时，则  $x*x=(2^{16}-2)*(2^{16}-2)=2^{32}-2*2*2^{16}+4 \pmod{2^{32}}=-(2^{18}-4)=-262140$ 。  
 $x$  的机器数为  $0000\text{FFFEH}$ ， $x*x$  的机器数为  $\text{FFFC}0004\text{H}$ 。

(2)  $(x-1 < 0) \parallel x > 0$

非永真。当  $x=-2\ 147\ 483\ 648$  时，显然， $x < 0$ ，机器数为  $80000000\text{H}$ ， $x-1$  的机器数为  $7\text{FFFFFFFH}$ ，符号位为 0，因而  $x-1 > 0$ 。此时， $(x-1 < 0)$  和  $x > 0$  两者都不成立。

(3)  $x < 0 \parallel -x \leq 0$

永真。若  $x > 0$ ， $x$  符号位为 0 且数值部分为非 0（至少有一位是 1），从而使  $-x$  的符号位一定是 1，即则  $-x < 0$ ；若  $x=0$ ，则  $-x=0$ 。综上，只要  $x < 0$  为假，则  $-x \leq 0$  一定为真，因而是永真。

(4)  $x > 0 \parallel -x \geq 0$

非永真。当  $x=-2\ 147\ 483\ 648$  时， $x < 0$ ，且  $x$  和  $-x$  的机器数都为  $80000000\text{H}$ ，即  $-x < 0$ 。此时， $x > 0$  和  $-x \geq 0$  两者都不成立。

(5)  $x \& 0xf \neq 15 \parallel (x < 28) < 0$

非永真。这里  $\neq$  的优先级比  $\&$ （按位与）的优先级高。因此，若  $x=0$ ，则  $x \& 0xf \neq 15$  为 0， $(x < 28) < 0$  也为 0，所以结果为假。

(6)  $x > y == (-x < -y)$

非永真。当  $x=-2\ 147\ 483\ 648$ 、 $y$  任意（除  $-2\ 147\ 483\ 648$  外），或者  $y=-2\ 147\ 483\ 648$ 、 $x$  任意（除  $-2\ 147\ 483\ 648$  外）时不等。因为  $\text{int}$  型负数  $-2\ 147\ 483\ 648$  是最小负数，该数取负后结果仍为  $-2\ 147\ 483\ 648$ ，而不是  $2\ 147\ 483\ 648$ 。

(7)  $\sim x + \sim y == \sim(x+y)$

永假。 $[-x]_{\text{补}} = \sim[x]_{\text{补}} + 1$ ， $[-y]_{\text{补}} = \sim[y]_{\text{补}} + 1$ ，故  $\sim[x]_{\text{补}} + \sim[y]_{\text{补}} = [-x]_{\text{补}} + [-y]_{\text{补}} - 2$ 。

$[-(x+y)]_{\text{补}} = \sim[x+y]_{\text{补}} + 1$ ，故  $\sim[x+y]_{\text{补}} = [-(x+y)]_{\text{补}} - 1 = [-x]_{\text{补}} + [-y]_{\text{补}} - 1$ 。

由此可见，左边比右边少 1。

(8)  $(\text{int})(ux-uy) == -(y-x)$

永真。 $(\text{int}) ux-uy = [x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} = [-y+x]_{\text{补}} = [-(y-x)]_{\text{补}}$

(9)  $((x >> 2) < 2) \leq x$

永真。因为右移总是向负无穷大方向取整。

(10)  $x*4 + y*8 == (x < 2) + (y < 3)$

永真。因为带符号整数  $x$  乘以  $2^k$  完全等于  $x$  左移  $k$  位，无论结果是否溢出。

(11)  $x/4+y/8==(x>>2)+(y>>3)$

非永真。当  $x=-1$  或  $y=-1$  时， $x/4$  或  $y/8$  等于 0，但是，因为  $-1$  的机器数为全 1，所以， $x>>2$  或  $y>>3$  还是等于  $-1$ 。此外，当  $x$  或  $y$  为负数且  $x$  不能被 4 整除或  $y$  不能被 8 整除，则  $x/4$  不等于  $x>>2$ ， $y/8$  不等于  $y>>3$ 。

(12)  $x*y==ux*uy$

永真。根据第 2.7.5 节内容可知， $x*y$  的低 32 位和  $ux*uy$  的低 32 位是完全一样的位序列。

(13)  $x+y==ux+uy$

永真。根据第 2.7.4 节内容可知，带符号整数和无符号整数都是在同一个整数加减运算部件中进行运算的， $x$  和  $ux$  具有相同的机器数， $y$  和  $uy$  具有相同的机器数，因而  $x+y$  和  $ux+uy$  具有完全一样的位序列。

(14)  $x*\sim y+ux*uy==\sim x$

永真。 $\sim y=\sim y+1$ ，即  $\sim y=-y-1$ 。而  $ux*uy=x*y$ ，因此，等式左边为  $x*(-y-1)+x*y=-x$ 。

35. 变量  $dx$ 、 $dy$  和  $dz$  的声明和初始化如下：

```
double dx = (double) x;
```

```
double dy = (double) y;
```

```
double dz = (double) z;
```

若  $float$  和  $double$  分别采用 IEEE 754 单精度和双精度浮点数格式， $sizeof(int)=4$ ，则对于任意  $int$  型变量  $x$ 、 $y$  和  $z$ ，判断以下关系表达式是否永真。若永真则给出证明；若不永真则给出结果为假时  $x$  和  $y$  的取值。

(1)  $dx*dx >= 0$

(2)  $(double)(float) x == dx$

(3)  $dx+dy == (double) (x+y)$

(4)  $(dx+dy)+dz == dx+(dy+dz)$

(5)  $dx*dy*dz == dz*dy*dx$

(6)  $dx/dx == dy/dy$

参考答案：

(1)  $dx*dx >= 0$

永真。 $double$  型数据用 IEEE 754 标准表示，尾数用原码小数表示，符号和数值部分分开运算。不管结果是否溢出都不会影响乘积的符号。

(2)  $(double)(float) x == dx$

非永真。当  $int$  型数据  $x$  的有效位数比  $float$  型可表示的最大有效位数 24 更多时， $x$  强制转换为  $float$  型数据时有效位数丢失，而将  $x$  转换为  $double$  型数据时没有有效位数丢失。也即等式左边可能是近似值，而右边是精确值。

(3)  $dx+dy == (double) (x+y)$

非永真。因为  $x+y$  可能会溢出，而  $dx+dy$  不会溢出。

(4)  $(dx+dy)+dz == dx+(dy+dz)$

永真。因为  $dx$ 、 $dy$  和  $dz$  是由 32 位  $int$  型数据转换得到的，而  $double$  类型可以精确表示  $int$  类型数据，并且对阶时尾数移位位数不会超过 52 位，因此尾数不会舍入，因而不会发生大数吃小数的情况。

但是，如果  $dx$ 、 $dy$  和  $dz$  是任意  $double$  类型数据，则非永真。

(5)  $dx*dy*dz == dz*dy*dx$

非永真。相乘的结果可能产生舍入。

(6)  $dx/dx == dy/dy$

非永真。 $dx$  和  $dy$  中只要有一个为 0、另一个不为 0 就不相等。